

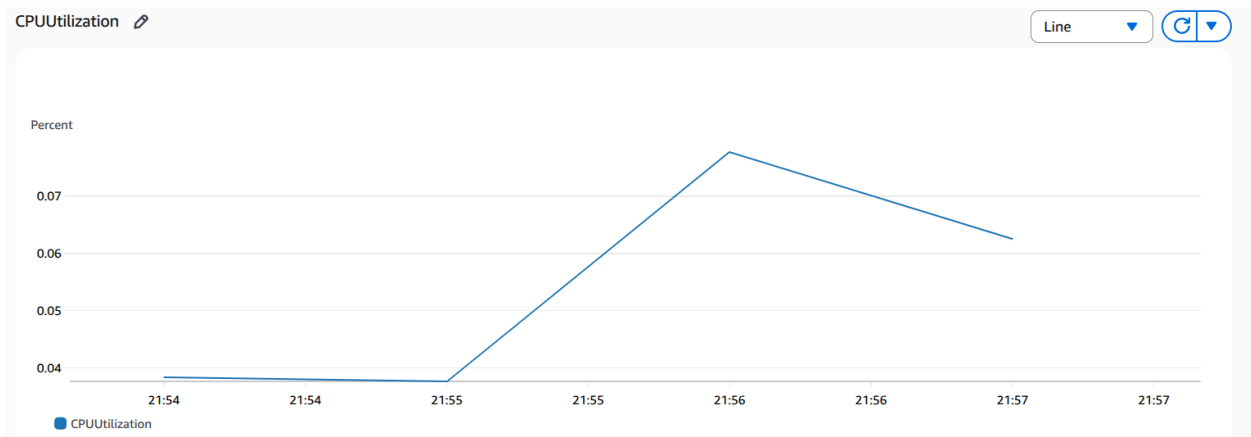
Test 1 - Baseline health check test

Description: Tests /admin/health endpoint with a load pattern of 1 user, constant rate of requests for 60s

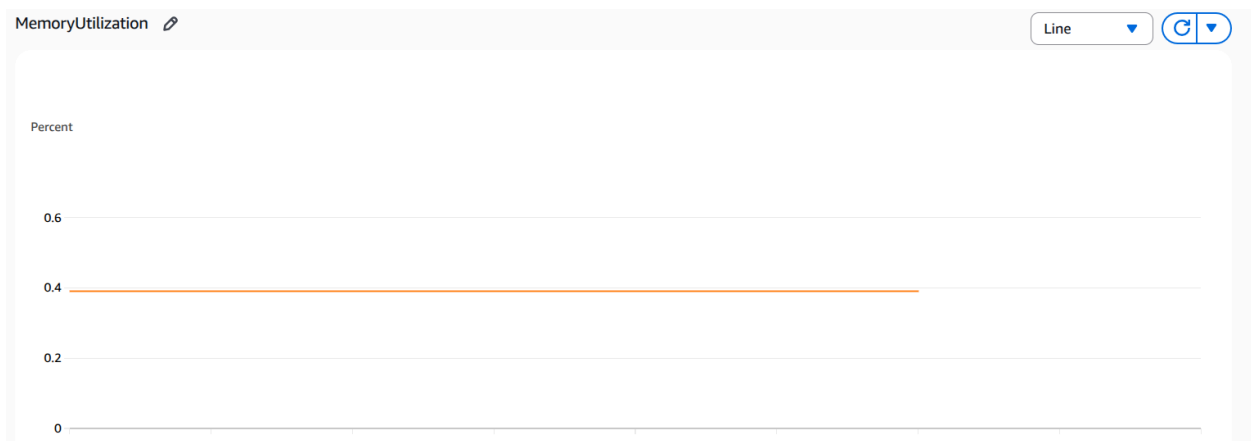
Ideal expectation:

- Response time is consistently low
- All dependencies report healthy status

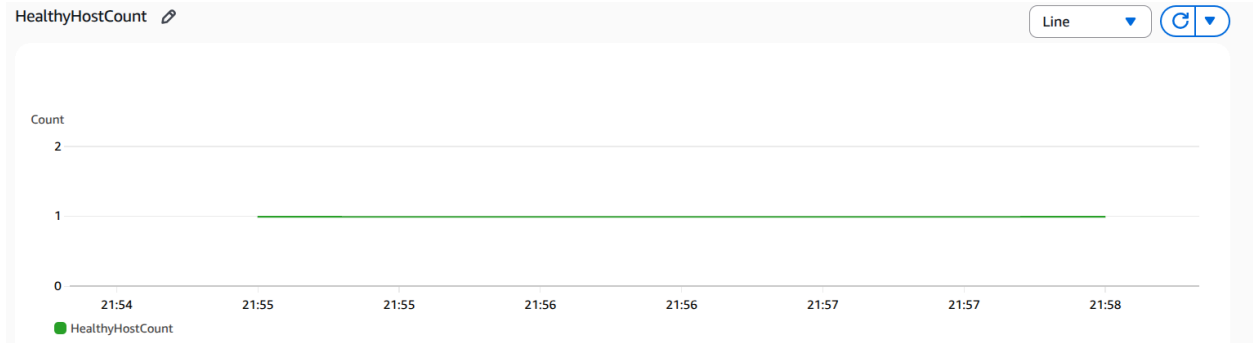
ECS CPU Utilization: Low



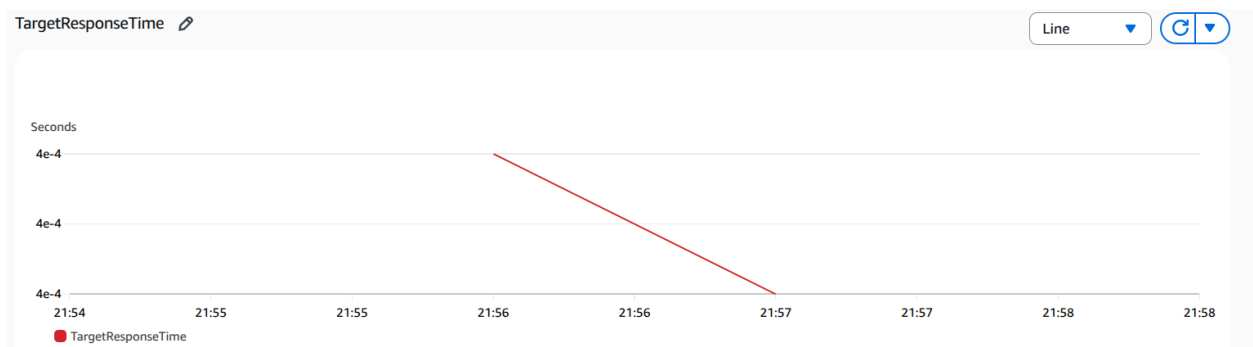
ECS Memory Utilization: Low



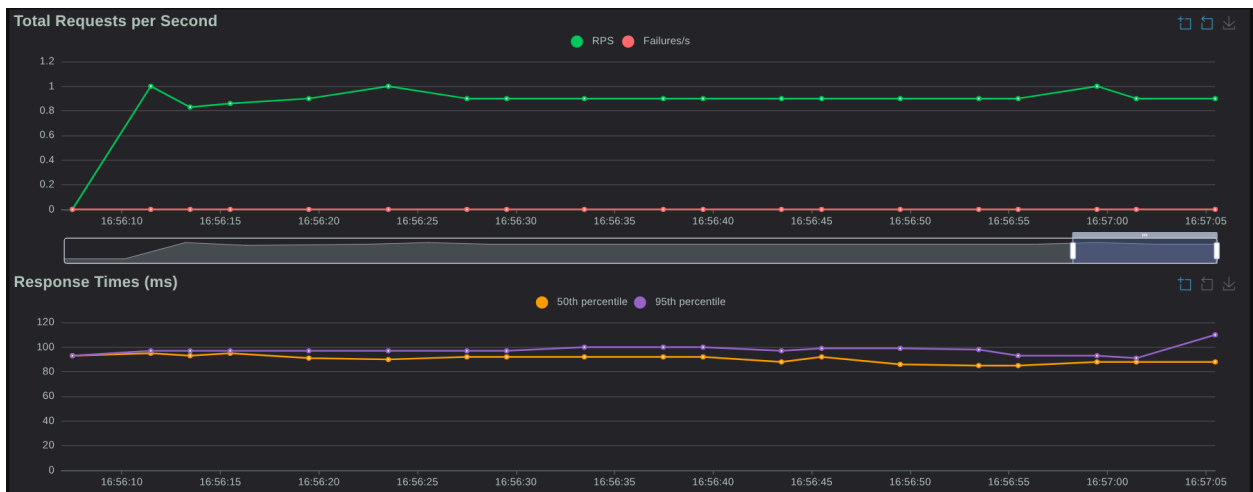
ALB Healthy Host Count: Consistent at 1



ALB Target response time: Consistently low



Locust Chart: Response times are consistently low, minor spike in p95 response time going up over 100ms at the end



Test 2 - Validate basic payment creation flow without load

Description: Tests POST /payments with valid payload with idempotency key header enforced. Load pattern is 10 users for constant request rate of 5 minutes

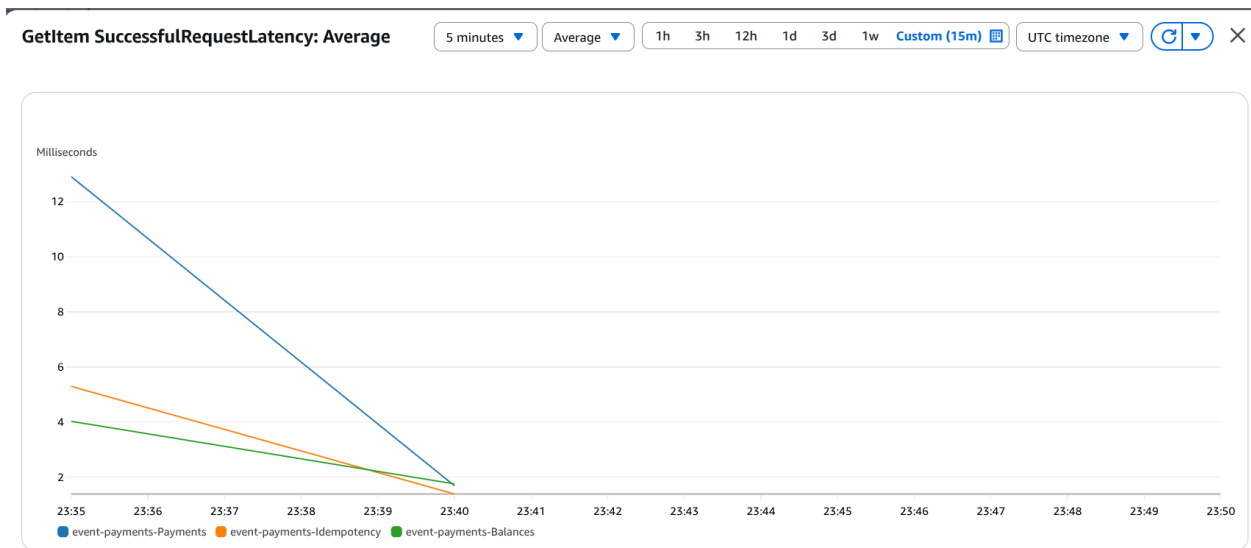
Ideal expectation:

- 100% success rate
- Good P95 response time
- Good DynamoDB latency

P95 response times are good for all operations except Initial create payment for all 10 users, further create payments all have acceptable response times

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Create Payment	434	0	110	140	860	119.86	90	1225	53	2.5	0
POST	Create Payment (Setup)	10	0	280	1400	1400	413.71	221	1417	53	0	0
GET	Get Balance	158	0	98	140	200	107.48	82	1201	125.3	1	0
GET	Get Payment	252	0	99	120	160	100.15	82	232	123	1.6	0
	Aggregated	854	0	100	140	400	115.19	82	1417	87.03	5.1	0

GetItem and PutItem latencies start high but reduce gradually likely due to DynamoDB's adaptive capacity wamup



PutItem SuccessfulRequestLatency: Average

5 minutes

Average

1h

3h

12h

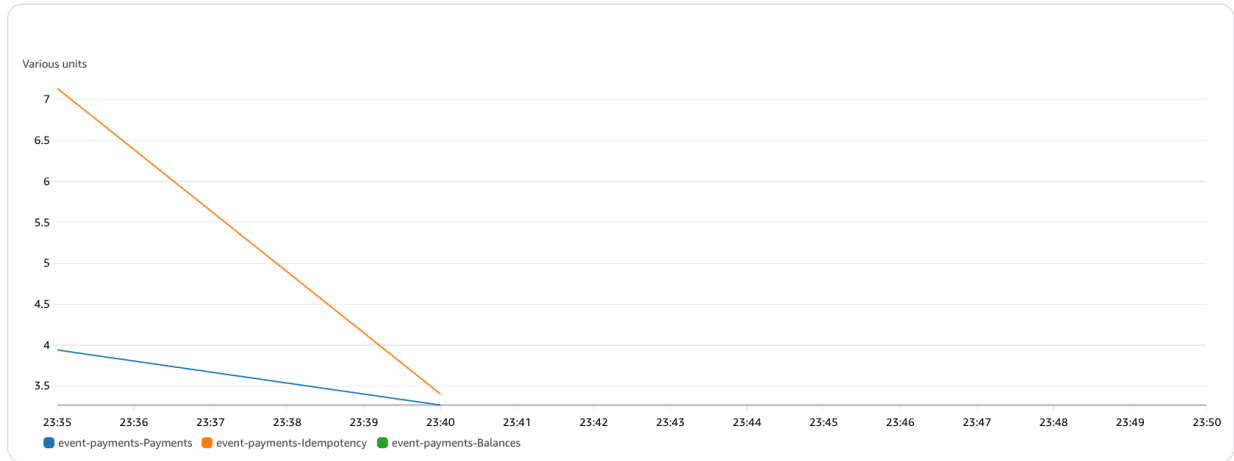
1d

3d

1w

Custom (15m)

UTC timezone



No user errors or requests throttled

UserErrors: Sum

5 minutes

Sum

1h

3h

12h

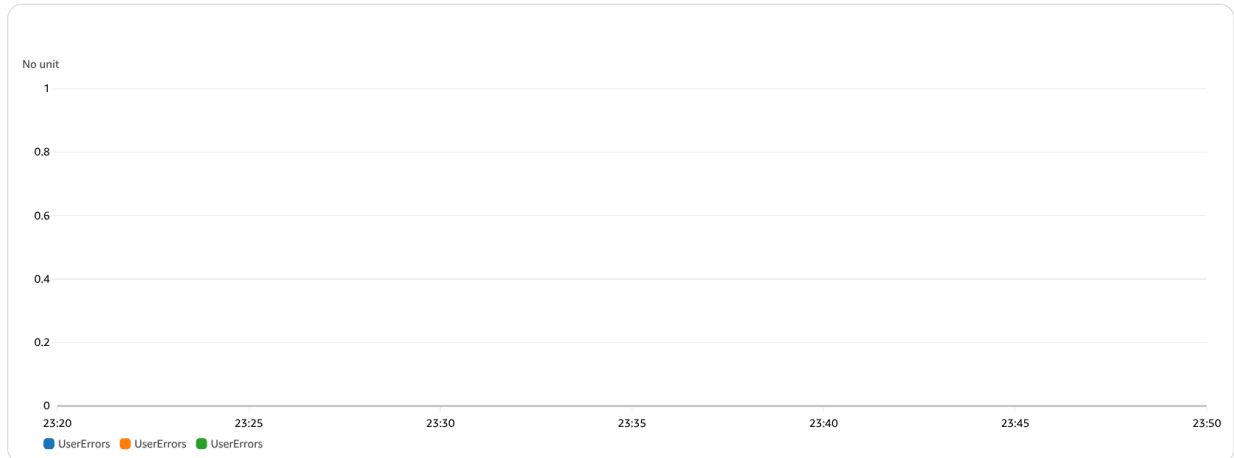
1d

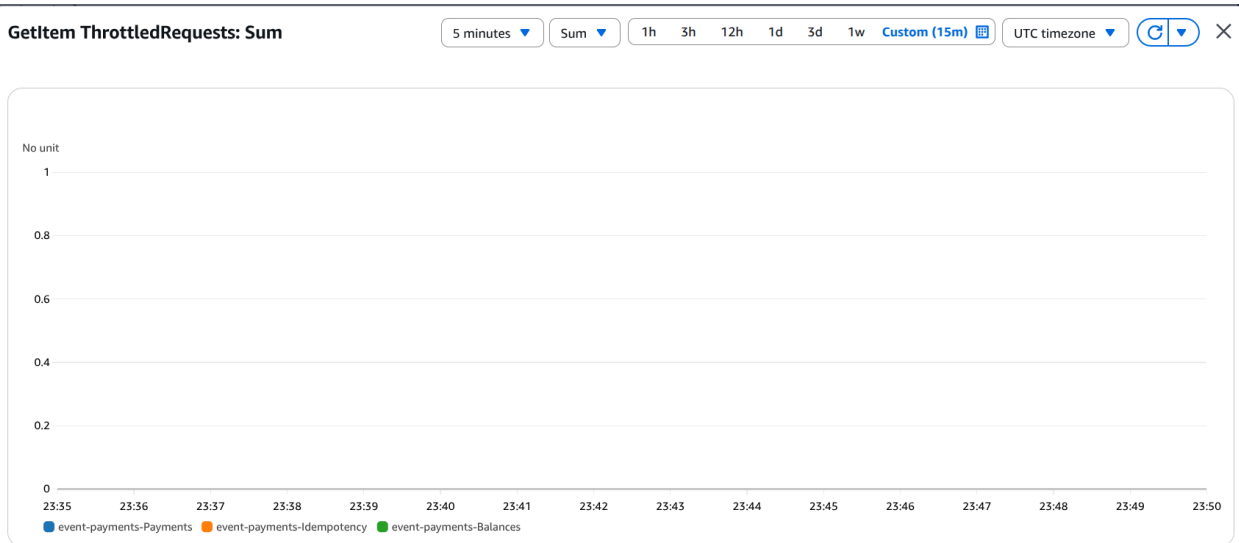
3d

1w

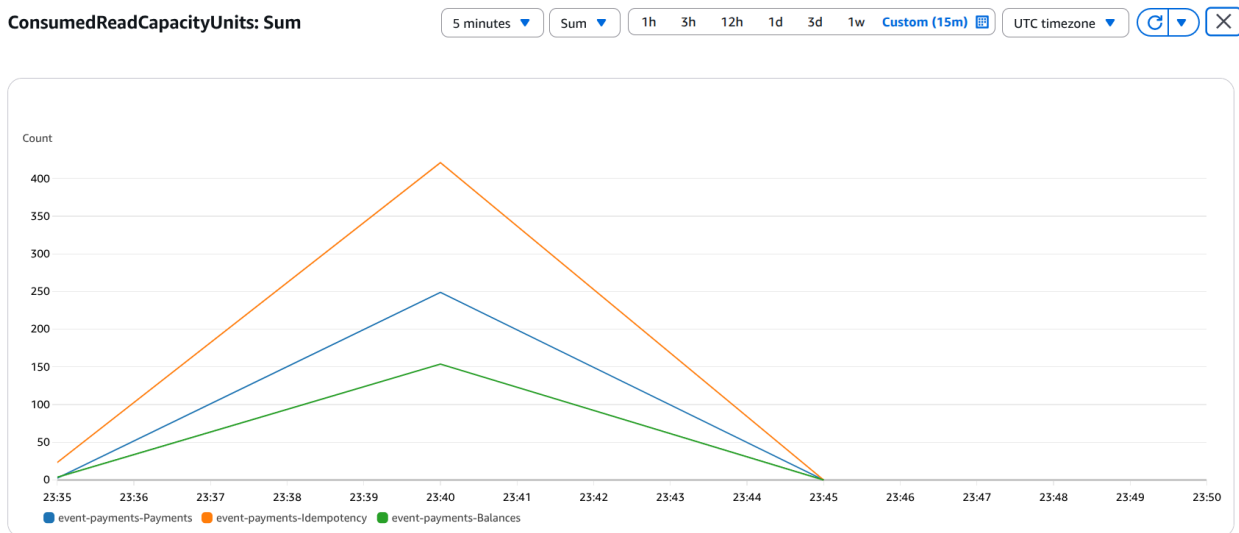
Custom (30m)

UTC timezone

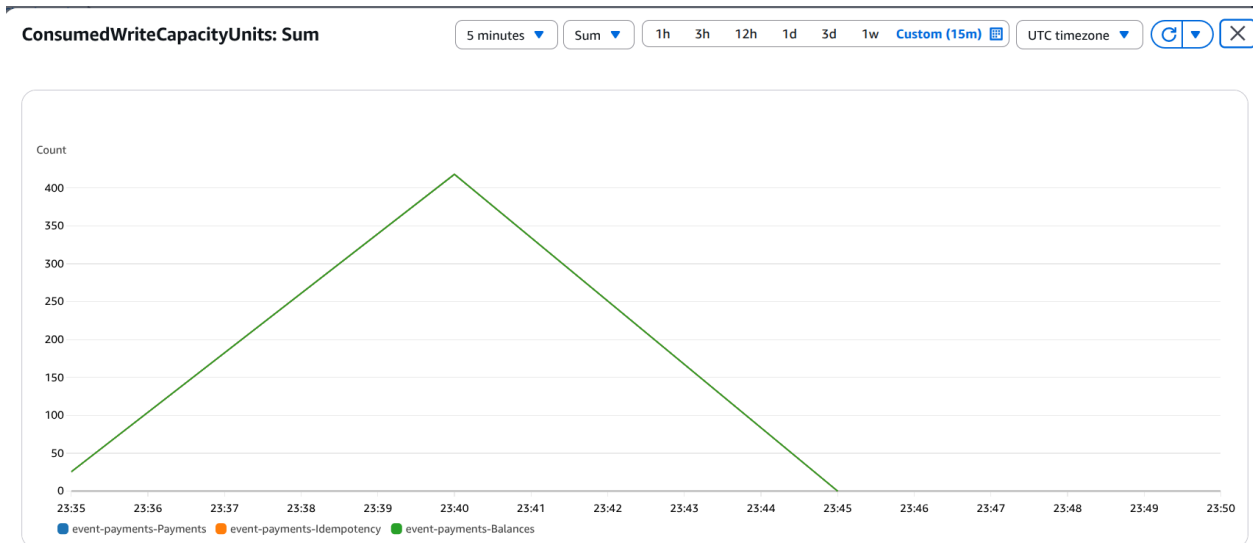




Total of 820RCUs consumed, with the highest consumer being the Idempotency table, consuming 420RCUs. Idempotency table is the highest consumer because every single request checks idempotency first.



Not visible in this chart but all tables consume the exact same number of WCUs, peaking at 400.



Test 3: Idempotency test

Description: With a load pattern of 50 concurrent users, spawning all at once, each making 10 requests simultaneously it should verify idempotency guarantees.

Ideal expectation:

- The idempotency table has exactly 50 entries
- No request failures
- No ConditionalCheckFailedRequests

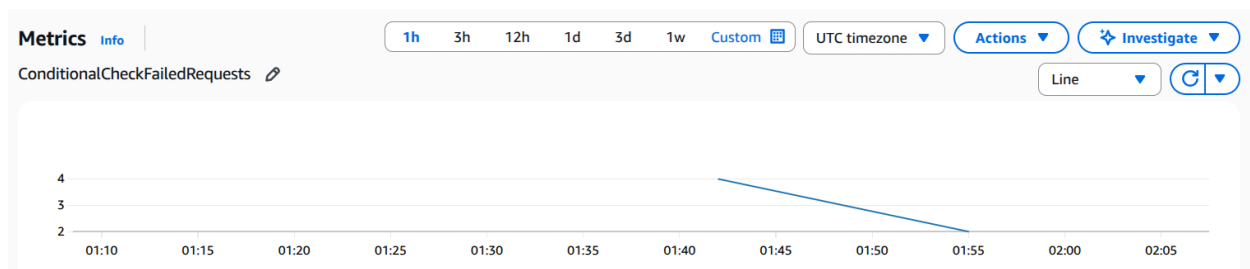
Only 29 items in idempotency table

```
swapnil@Kilgore:~/NEU/cs6650/Distributed-Payment-Logger/terraform$ aws dynamodb scan --table-name event-payments-Idempotency --select COUNT
{
  "Count": 29,
  "ScannedCount": 29,
  "ConsumedCapacity": null
}
swapnil@Kilgore:~/NEU/cs6650/Distributed-Payment-Logger/terraform$ aws dynamodb scan --table-name event-payments-Payments --select COUNT
{
  "Count": 31,
  "ScannedCount": 31,
  "ConsumedCapacity": null
}
```

No request failures

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Duplicate Key (Different Body)	2354	0	100	140	170	109.05	83	586	53	41.2	0
POST	Duplicate Request (Same Body)	23878	0	100	140	190	109.89	81	670	53	413.8	0
	Aggregated	26232	0	100	140	190	109.81	81	670	53	455	0

Multiple ConditionalCheckFailedRequests



The application currently does not meet the success criteria for this test, while the API is processing the request and returning a 202 response, there are conditionalcheckfailed requests due to race conditions caused by 50 users spawning at once.

Future work: Better handle race conditions to deal with DynamoDB's conditional check fails, ideally all writes to the idempotency table should be successful even under load.

Test 4: Baseline Load test

Description: A baseline load test of 100users, ramping up at 1 user per second runs for 10 minutes. Over this duration all execute an entire payment workflow Create->Authorize->Capture->Verify(balance).

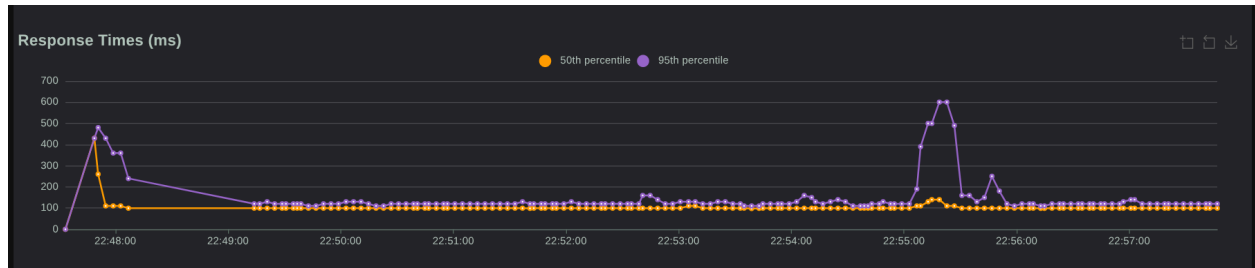
Ideal expectation:

- No failures observed
- No DynamoDB user errors or throttled requests
- An examination of system resource usage patterns

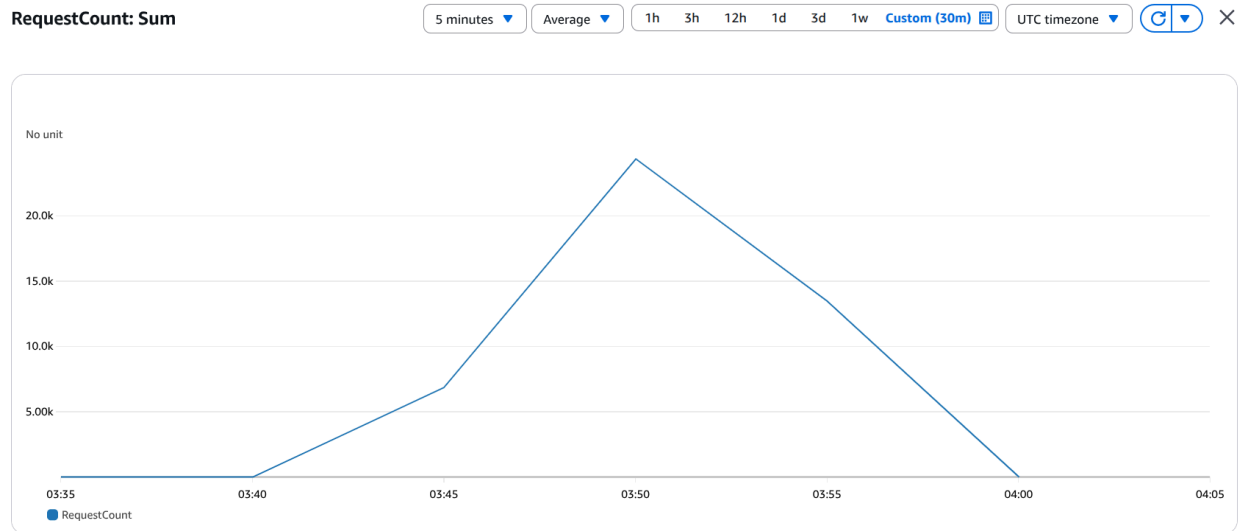
Locust stats: No failures observed, good P95 response times all under 150ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /accounts/id/balance	11153	0	100	120	180	103.2	80	844	140.24	20.6	0
POST	POST /payments (Create)	11221	0	110	140	320	113.06	85	916	53	20.2	0
POST	POST /payments/id/authorize	11186	0	100	120	220	104.93	81	835	0	20.1	0
POST	POST /payments/id/capture	11153	0	100	120	230	104.76	81	777	0	20.4	0
Aggregated		44713	0	100	130	250	106.5	80	916	48.28	81.3	0

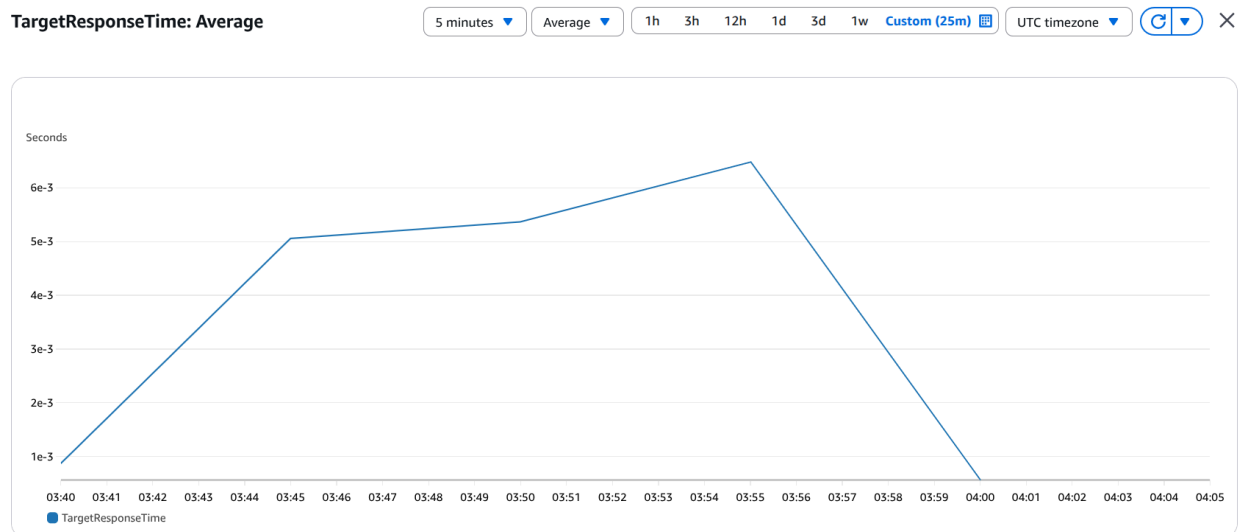
Locust chart: Response times remain stable apart from the cold start during the ramp up at the beginning when creating payments for the first time and the spike later which is likely due to DynamoDB's adaptive capacity adjustment.



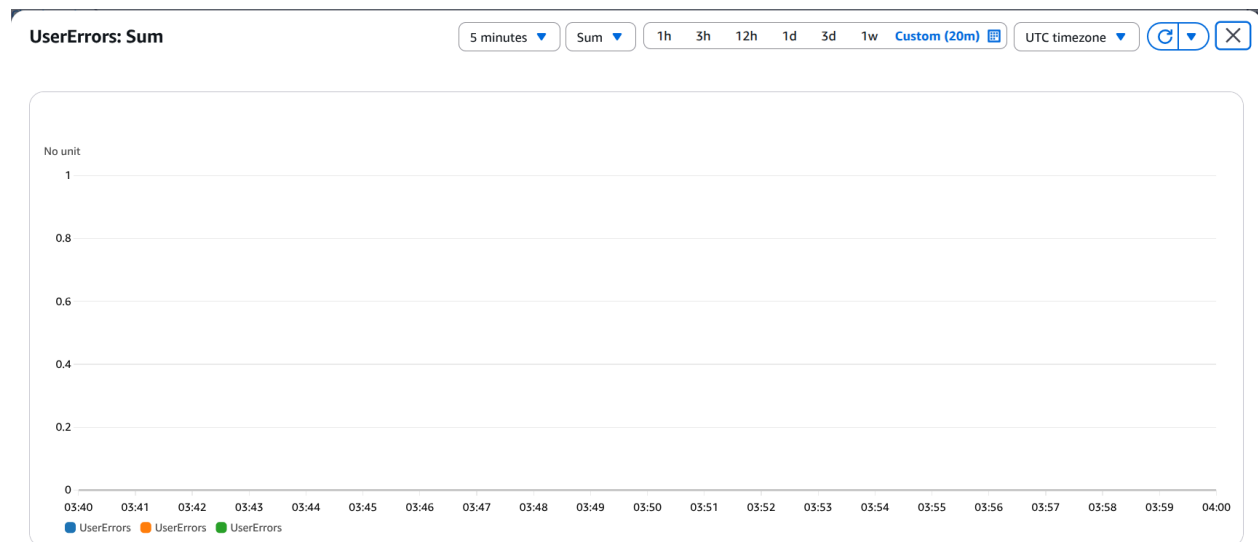
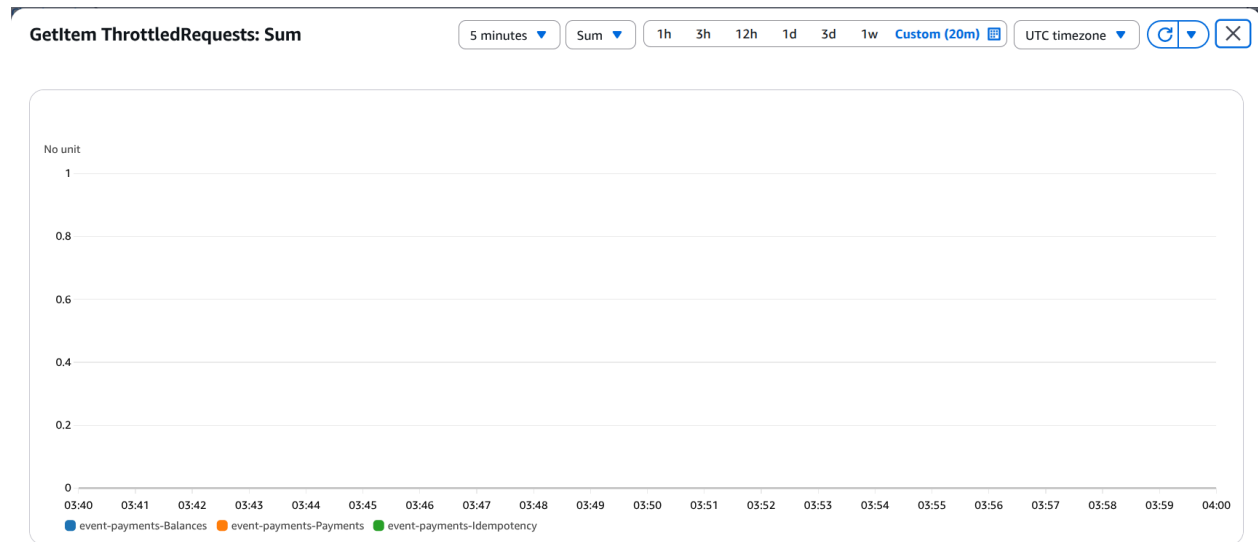
Cloudwatch ALB requestcount chart matches the test load. Traffic went to a peak of 23k requests



As load increased, ALB's response times degraded but not too radically, went from 1ms to 5ms after ramp up and stayed relatively flat while increasing to 6ms



No DynamoDB requests throttled or suffered UserErrors



Payments table consumed the most writes (18k peak) because each workflow creates a payment record, then updates it twice for authorization and capture states. This means 3 write operations per payment (initial create + authorize update + capture update).

Balances consumed medium writes (12k peak) because each payment triggers a balance debit operation on the account. One write per payment to update the account balance, plus occasional balance checks that might update metadata, resulting in ~1-1.5 writes per payment workflow.

Idempotency consumed the least writes (6k peak) because each unique payment only writes once to create the idempotency record. The test uses unique idempotency keys per request (idem-{random}), there are no duplicate requests, so exactly 1 write per payment - the lowest of all tables.

ConsumedWriteCapacityUnits: Sum

5 minutes

Sum

1h

3h

12h

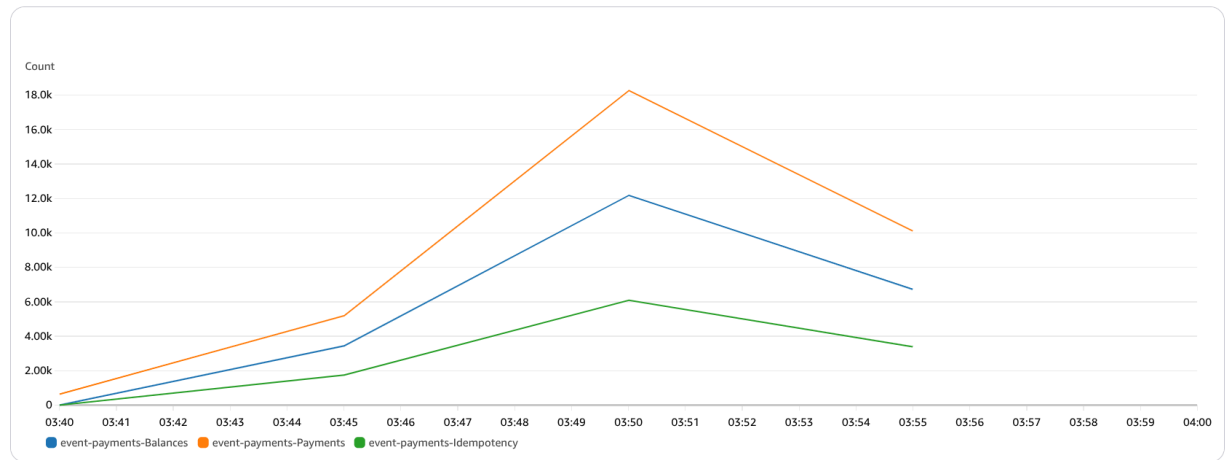
1d

3d

1w

Custom (20m)

UTC timezone



ECS memory utilization stayed very low at ~2%

MemoryUtilization: Average

5 minutes

Average

1h

3h

12h

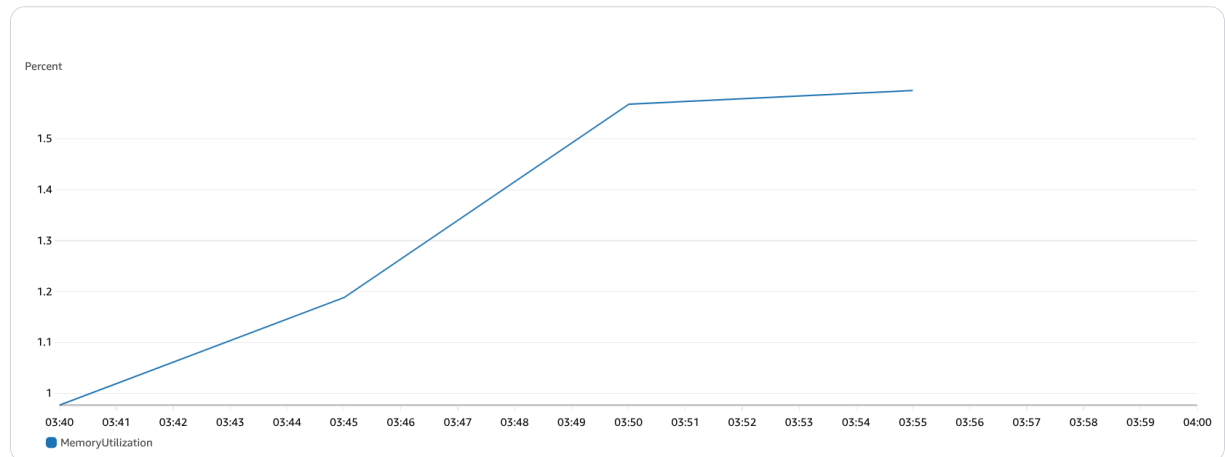
1d

3d

1w

Custom (20m)

UTC timezone



CPU utilization peaked at 28%. Since the scaling event is triggered only when the CPU util hits 70%, no autoscaling event was triggered.

